NAAN MUDHALVAN | SWTID-2026-4524

Implement Client Script & UI Policy (Incident)

Project team | Team ID SWTID-2026-4524 | 2 October 2026

SR **Swathi R** — Team Leader
SD **Sandhiya D** — Member
SC **Sandhya Charlet Calista S** — Member
MD **Mallemkonda Dallyjasmine** — Member

SCENARIO

Configuring, scripting and testing field controls on the ServiceNow Incident form

	Identify	Configure	Script	Test	Extend
	What problem starts the work?	What is set up with the UI Policy?	What do the Client Scripts do?	How is each rule verified?	What happens after the build?
S Steps What does the person (or team) typically do?	Notice incomplete or inconsistent Incident data Review the problem statement and objective Decide the business rule: High impact needs extra controls Choose UI Policies and Client Scripts as the solution	Log in with admin or configuration access Go to System UI > UI Policies and click New Create 'High Impact Control' on Incident, Active = true Condition: Impact is 1 - High; make Assignment group mandatory; Reverse if false = true Add UI Policy Action: Urgency set to Read-only	Go to System UI > Client Scripts and click New onChange 'Auto set urgency for high impact': Impact changes to High, Urgency set to High, info message shown onSubmit 'Prevent save if Assigned To missing': error and no save when Impact is High and Assigned To is empty onCellEdit 'Prevent state change via list edit': alert shown and change blocked	Test mandatory enforcement: High impact, Assigned To empty Test successful save with Assigned To filled Reverse condition test: change Impact from High to Medium Test list edit blocking on State Test form-based State update	Incident records meet the defined business rules Write the conclusion and share the micro project
I Interactions What interactions happen at each step? People, places and things	People: team members Places: ServiceNow instance Things: Incident records, problem statement	Places: System UI > UI Policies Things: UI Policy record, Conditions section, UI Policy Actions related list	Places: System UI > Client Scripts Things: script editor, Impact, Urgency, Assigned To and State fields, g_form	Places: Incident > Create New, Incident > Open, Incident > All (list view) Things: Incident form, Incident list, error and alert messages	People: team and reviewers Things: project documentation and results
G Goals & motivations At each step, what is the primary goal? ('Help me...' or 'Help me avoid...')	Help me improve Incident data quality Help me avoid wrong or missing data affecting reporting and SLA compliance	Help me apply extra controls only when Impact is High Help me make sure fields go back to normal when the condition is not met	Help me update Urgency automatically Help me prevent saving when required data is missing Help me avoid unwanted State changes from the list	Help me confirm every rule works and reverts correctly Help me avoid releasing a rule that fails a scenario	Help me show the value of client-side controls Help me reuse this approach later
P Positive moments What steps are enjoyable, productive or satisfying?	The problem and objective are clear and easy to explain	Reverse if false restores fields automatically The UI Policy is simple to configure	Urgency updates on its own when Impact is High Clear error message appears on Assigned To	Incident saves successfully once Assigned To is filled Rules switch off correctly when Impact changes to Medium	Cleaner, more consistent data across the platform Lightweight and easy to implement
N Negative moments What steps are frustrating, confusing or time-consuming?	Relying on user awareness and manual checks leads to inconsistent data	Admin or configuration access is needed Conditions must be set exactly right for the policy to trigger	Script condition must match the UI Policy condition A typo in a script can change save behaviour The list edit alert interrupts the user	Several scenarios must be tested to cover every rule	Client-side rules apply only in the user interface
O Areas of opportunity How might we make each step better? What ideas do we have?	How might we find more fields that need validation?	How might we apply similar controls for other impact levels?	How might we align the mandatory field in the UI Policy and the onSubmit script? Could we explain why Urgency is read-only?	Could we keep a reusable test checklist for every rule?	How might we apply the same pattern to other tables? Could a server-side check back up the client-side rules?